



En esta clase

- El modelo computacional de la POO
- El diagrama de una clase
- La implementación de una clase en Java
- Los atributos de clase y de instancia
- Los servicios: constructores, comandos y consultas
- Ejemplo: Cajero automático
- La clase tester

Problemas y soluciones



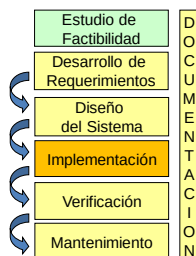
Un problema de pequeña escala puede ser modelado por un **programa** desarrollado por un **programador** a través de un **proceso de abstracción** enfocado en la representación de los datos y el diseño de los algoritmos.

Problemas y soluciones

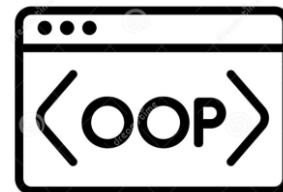


Un problema de mediana o gran escala requiere ser modelado por un **sistema de software** desarrollado por un **equipo de profesionales** a través de un **proceso de desarrollo** en el que se aplica una **metodología**.

El proceso de desarrollo de software



El paradigma de programación orientada a objetos



El concepto de objeto

- El término **objeto** se utiliza para referirse a dos conceptos relacionados pero diferentes.
- Durante el desarrollo de requerimientos y el diseño del sistema se **identifican los objetos del problema**.
- Durante la ejecución del sistema que modela el problema se **crean objetos de software**.
- Cada **objeto del problema** está asociado a un **objeto de software** que lo representa en ejecución.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

El concepto de clase

- Durante el diseño de un sistema una **clase** es un patrón que establece los **atributos** y el **comportamiento** de un conjunto de objetos.
- Un **atributo** es una propiedad o cualidad relevante que caracteriza a todos los objetos de una clase.
- El **comportamiento** queda determinado por el conjunto de **servicios** que brinda la clase y las **responsabilidades** que asume.
- En la implementación de un sistema una **clase** es un **módulo de software** que puede ser desarrollado con alguna independencia del resto de los demás módulos.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

El desarrollo de requerimientos

El **ajedrez** es un juego de mesa para dos personas. Es uno de los juegos más populares del mundo. Se considera no sólo un juego, sino un arte, una ciencia y un deporte mental. Cada bando posee 16 **piezas**, con diferentes capacidades de movimiento, que se mueven en un **tablero** cuadrado de 8x8 casillas, alternativamente claras y oscuras. Las piezas de cada jugador al principio de la partida son un rey, una dama o reina, dos alfiles, dos caballos, dos torres y ocho peones.



El desarrollo de requerimientos

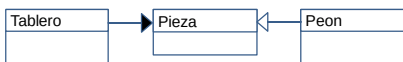
En esta etapa se **especifica los requerimientos** y se elabora un **modelo del problema** a partir de la identificación de los **objetos** relevantes y de los **atributos** que permiten agruparlos en **clases**.



El diseño orientado a objetos

- Durante el diseño se identifica el comportamiento de los objetos y se completa la especificación de las clases.
- La estrategia sigue siendo **dividir para conquistar** pero los módulos no son subprogramas o conjuntos de subprogramas, sino clases.
- El resultado de esta etapa es un conjunto de diagramas que **modelan la solución** en un **lenguaje de modelado**.
- Entre ellos el **diagrama de clases** especifica la **colección de clases** y sus **relaciones**.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```



La implementación

- La implementación de un sistema orientado a objetos consiste en escribir el código en un **lenguaje de programación**.
- Cada clase modelada en el diagrama de clases se implementa en un **módulo de software**.
- La **implementación de cada servicio** puede constituir en sí mismo un problema y demandar el diseño de un **algoritmo** que lo resuelva.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

```

class Tablero{
}

class Pieza{
}

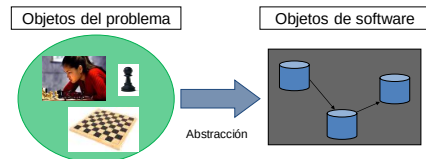
class Peon extends Pieza{
}
    
```

La verificación

- La **verificación de una clase** consiste en chequear que cada servicio funciona correctamente para un conjunto de casos de prueba.
- Si la clase Tablero **usa** los servicios provistos por la clase Peon, verificaremos cada servicio de la clase Peon y luego los servicios de la clase Tablero.
- La **verificación del sistema** consiste en chequear la integración de los módulos para un conjunto de casos de prueba.

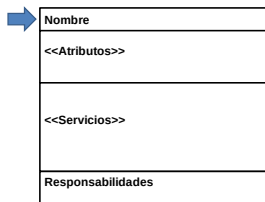
```
class Tester{
}
class Tablero{
}
class Pieza{
}
class Peon extends Pieza{
}
```

El modelo computacional



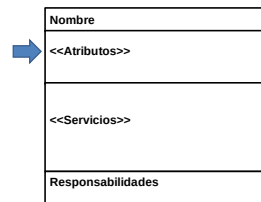
El **modelo computacional** de la POO es un mundo poblado de **objetos** comunicándose a través de **mensajes**. Cada objeto responde a un mensaje de acuerdo al comportamiento determinado por su clase.

El diagrama de una clase



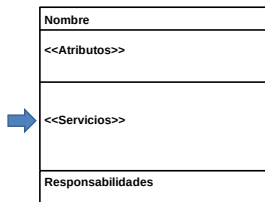
El **nombre** de una clase representa la abstracción del conjunto de instancias

El diagrama de una clase



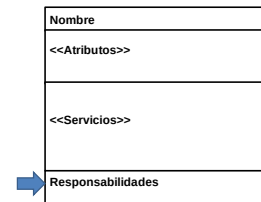
Un **atributo** es una propiedad o cualidad relevante que caracteriza a todos los objetos de una clase

El diagrama de una clase



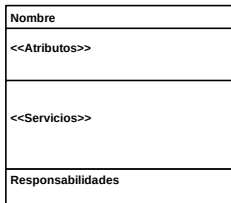
Un **servicio** es una operación que todas las instancias de una clase pueden realizar

El diagrama de una clase



Una **responsabilidad** representa un compromiso para la clase o un requerimiento.

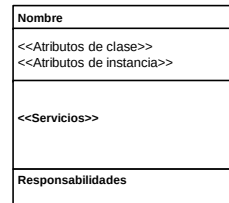
El diagrama de una clase



```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Las restricciones, requisitos y la funcionalidad de los servicios puede especificarse a través de notas o como un texto que acompaña al diagrama.

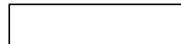
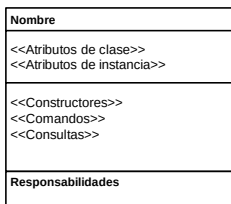
El diagrama de una clase



```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Los atributos pueden clasificarse en **atributos de clase** y **atributos de instancia**. Los atributos de clase comparten un mismo valor para todas las instancias.

El diagrama de una clase



```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Los servicios se clasifican en **constructores**, **comandos** y **consultas**. Los comandos y consultas son los **métodos** provistos por la clase.

La implementación en Java

- Cada clase modelada en el diagrama de clases durante la etapa de diseño, va a implementarse como una clase en Java.
- Es decir, la unidad básica de programación en Java es la **clase**.
- La **estructura** de una clase implementada también va a ser similar a la estructura del diagrama.
- Los **miembros** de una clase son:
 - Atributos** de instancia y de clase.
 - Servicios**, pueden ser constructores o métodos.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Ejemplo: Cajero automático

Especificación de Requerimientos

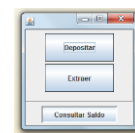
- Un banco ofrece **cajeros automáticos** a través de los cuales los clientes pueden **consultar el saldo** de su cuenta corriente y además realizar:
 - depósitos**,
 - extracciones**
- En el momento que se crea una cuenta corriente se establece su **código** y el **saldo** se inicializa en 0.
- También es posible crear una cuenta corriente estableciendo su código y saldo inicial.
- El código no se modifica, el saldo cambia con cada depósito o extracción. Una cuenta bancaria puede tener un saldo negativo hasta un máximo establecido por el banco.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Ejemplo: Cajero automático

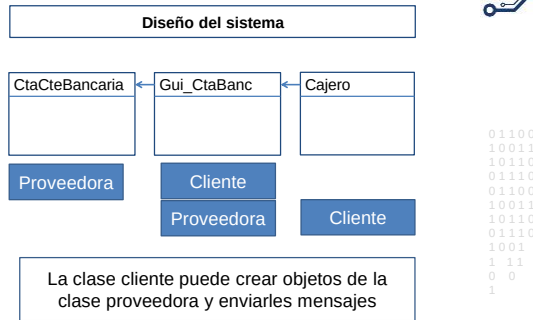
Especificación de Requerimientos

- Cada cliente accede a su cuenta corriente bancaria a través de una interfaz gráfica como la que sigue:

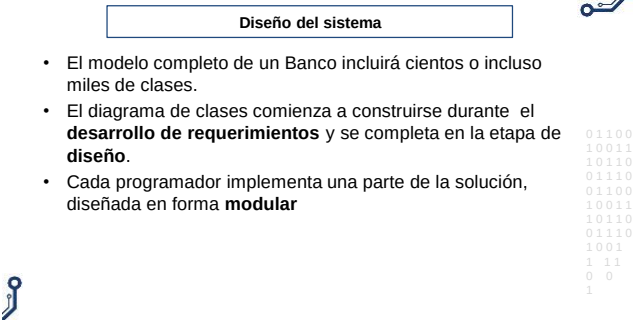


```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

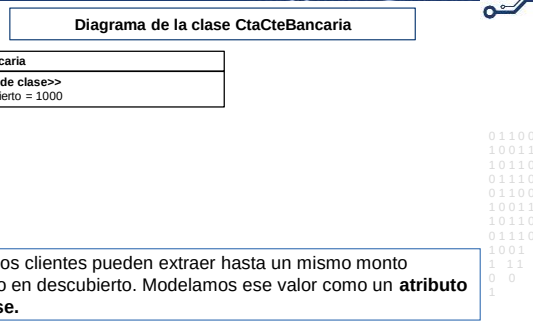
Ejemplo: Cajero automático



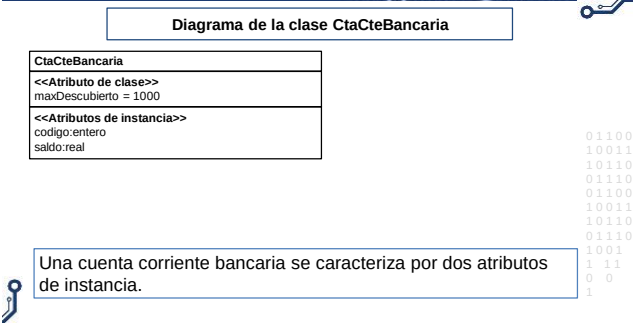
Ejemplo: Cajero automático



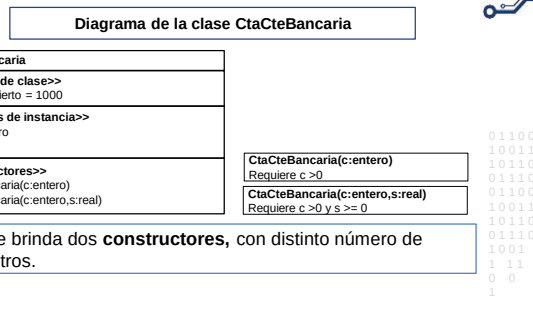
Ejemplo: Cajero automático



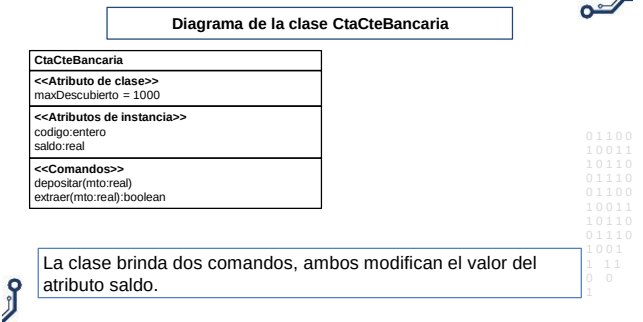
Ejemplo: Cajero automático



Ejemplo: Cajero automático

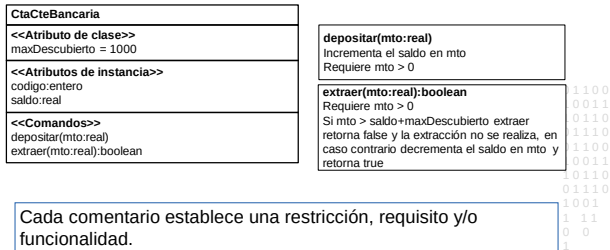


Ejemplo: Cajero automático



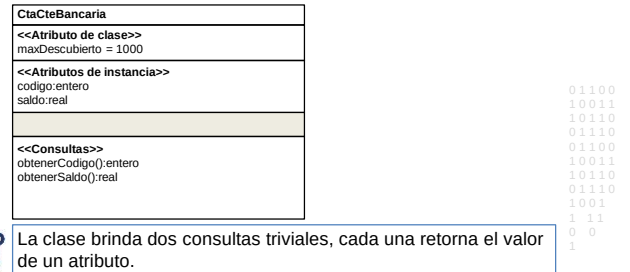
Ejemplo: Cajero automático

Diagrama de la clase CtaCteBancaria



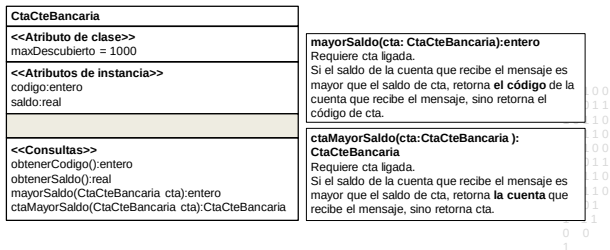
Ejemplo: Cajero automático

Diagrama de la clase CtaCteBancaria



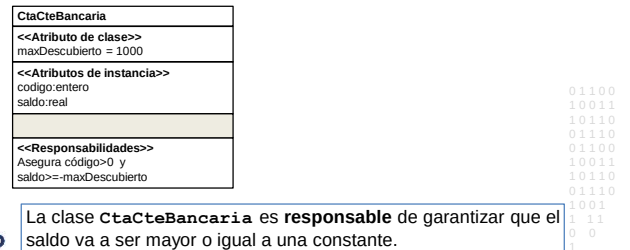
Ejemplo: Cajero automático

Diagrama de la clase CtaCteBancaria



Ejemplo: Cajero automático

Diagrama de la clase CtaCteBancaria



Ejemplo: Cajero automático

Decisiones de diseño

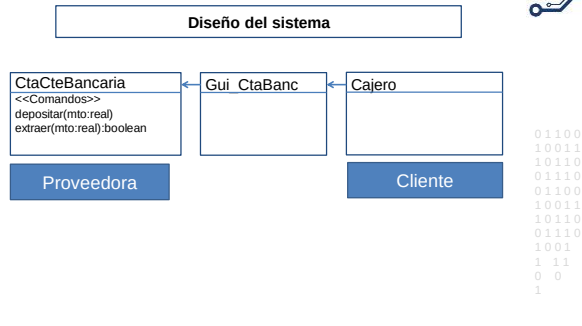
- Cada cuenta bancaria se caracteriza con dos atributos de instancia: código y saldo.
- El código tiene que ser estrictamente mayor a 0, el saldo debe ser mayor a una constante.
- En el momento que se crea una cuenta es indispensable establecer el código, que no va a cambiar.
- El saldo puede establecerse explícitamente al crearse una cuenta o queda inicializado en 0 implícitamente

Ejemplo: Cajero automático

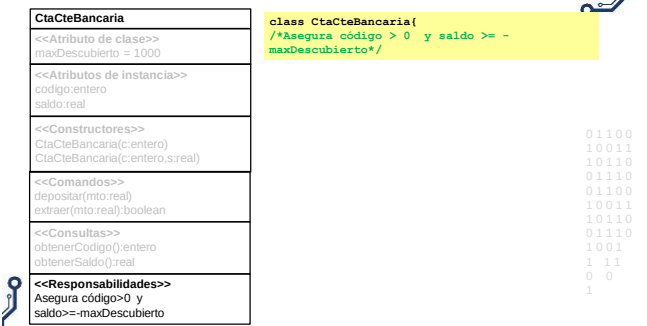
Decisiones de diseño

- Los comandos que modifican el saldo de la cuenta **requieren** que el parámetro **mto** contenga un valor mayor a 0.
- La clase **CtaCteBancaria** va a formar parte de una colección de clases.
- Toda clase que **use** a la clase **CtaCteBancaria** es responsable de garantizar que el parámetro real de **depositar** y **extraer** sea un valor mayor a cero.
- El comando **extraer** retorna un valor booleano indicando si la operación pudo realizarse.

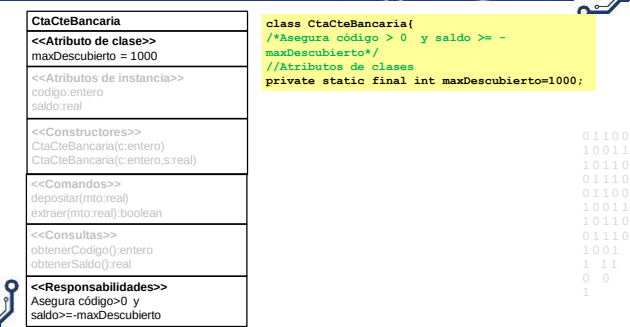
Ejemplo: Cajero automático



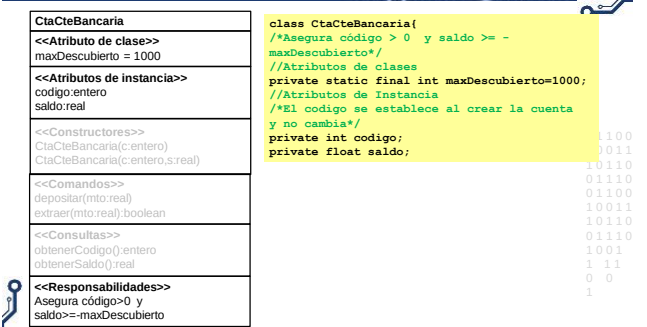
Ejemplo: Cajero automático



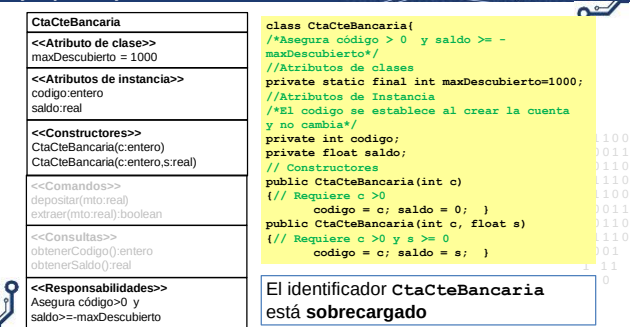
Ejemplo: Cajero automático



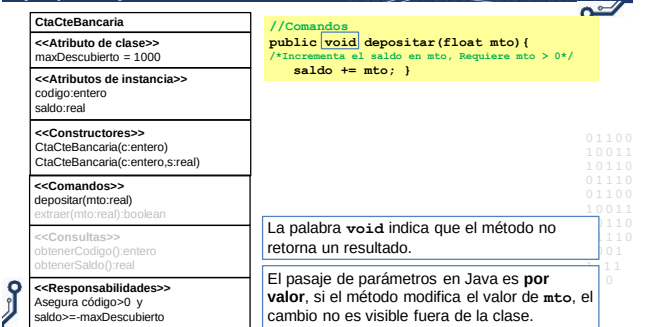
Ejemplo: Cajero automático



Ejemplo: Cajero automático



Ejemplo: Cajero automático



Introducción a la Programación Orientada a Objetos

Ejemplo: Cajero automático

CtaCteBancaria	
<<Atributo de clase>> maxDescubierto = 1000	
<<Atributos de instancia>> codigo:entero saldo:real	
<<Constructores>> CtaCteBancaria(c:entero) CtaCteBancaria(c:entero,s:real)	
<<Comandos>> depositar(mto:real) extraer(mto:real):boolean	
<<Consultas>> obtenerCodigo():entero obtenerSaldo():real	
<<Responsabilidades>> Asegura código>0 y saldo>=-maxDescubierto	

```
//Comandos
public void depositar(float mto){
    /*Incrementa el saldo en mto, Requiere mto > 0*/
    saldo += mto; }
```

El comentario del diagrama se mantiene en el código estableciendo la **responsabilidad** de la clase que **usa** a CtaCteBancaria.

La instrucción de asignación es equivalente a:
saldo = saldo + mto;

Ejemplo: Cajero automático

CtaCteBancaria	
<<Atributo de clase>> maxDescubierto = 1000	
<<Atributos de instancia>> codigo:entero saldo:real	
<<Constructores>> CtaCteBancaria(c:entero) CtaCteBancaria(c:entero,s:real)	
<<Comandos>> depositar(mto:real) extraer(mto:real):boolean	
<<Consultas>> obtenerCodigo():entero obtenerSaldo():real	
<<Responsabilidades>> Asegura código>0 y saldo>=-maxDescubierto	

```
//Comandos
public void depositar(float mto){
    /*Incrementa el saldo en mto, Requiere mto > 0*/
    saldo += mto; }

public boolean extraer(float mto){
    /*Requiere mto > 0. Si mto > saldo+maxDescubierto
    extraer retorna false y la extracción no se
    realiza, en caso contrario decrementa el saldo en
    mto y retorna true*/
    boolean puede = true;
    if (saldo + maxDescubierto >= mto)
        saldo=saldo-mto;
    else puede = false;
    return puede; }
```

La variable **puede** es local al método. Se crea en el momento que se ejecuta la declaración y se destruye al terminar la ejecución de **extraer**.

Ejemplo: Cajero automático

CtaCteBancaria	
<<Atributo de clase>> maxDescubierto = 1000	
<<Atributos de instancia>> codigo:entero saldo:real	
<<Constructores>> CtaCteBancaria(c:entero) CtaCteBancaria(c:entero,s:real)	
<<Comandos>> depositar(mto:real) extraer(mto:real):boolean	
<<Consultas>> obtenerCodigo():entero obtenerSaldo():real	
<<Responsabilidades>> Asegura código>0 y saldo>=-maxDescubierto	

```
//Comandos
public void depositar(float mto){
    /*Incrementa el saldo en mto, Requiere mto > 0*/
    saldo += mto; }

public boolean extraer(float mto){
    /*Requiere mto > 0. Si mto > saldo+maxDescubierto
    extraer retorna false y la extracción no se
    realiza, en caso contrario decrementa el saldo en
    mto y retorna true*/
    boolean puede = true;
    if (saldo + maxDescubierto >= mto)
        saldo=saldo-mto;
    else puede = false;
    return puede; }
```

El tipo del resultado es compatible con el tipo de la expresión que retorna.

Ejemplo: Cajero automático

CtaCteBancaria	
<<Atributo de clase>> maxDescubierto = 1000	
<<Atributos de instancia>> codigo:entero saldo:real	
<<Constructores>> CtaCteBancaria(c:entero) CtaCteBancaria(c:entero,s:real)	
<<Comandos>> depositar(mto:real) extraer(mto:real):boolean	
<<Consultas>> obtenerCodigo():entero obtenerSaldo():real	
<<Responsabilidades>> Asegura código>0 y saldo>=-maxDescubierto	

```
//Comandos
public void depositar(float mto){
    /*Incrementa el saldo en mto, Requiere mto > 0*/
    saldo += mto; }

public boolean extraer(float mto){
    /*Requiere mto > 0. Si mto > saldo+maxDescubierto
    extraer retorna false y la extracción no se
    realiza, en caso contrario decrementa el saldo en
    mto y retorna true*/
    boolean puede = true;
    if (saldo + maxDescubierto >= mto)
        saldo=saldo-mto;
    else puede = false;
    return puede; }
```

El comando **extraer** accede también a un atributo de clase y uno de instancia, **maxDescubierto** y **saldo**.

Ejemplo: Cajero automático

CtaCteBancaria	
<<Atributo de clase>> maxDescubierto = 1000	
<<Atributos de instancia>> codigo:entero saldo:real	
<<Constructores>> CtaCteBancaria(c:entero) CtaCteBancaria(c:entero,s:real)	
<<Comandos>> depositar(mto:real) extraer(mto:real):boolean	
<<Consultas>> obtenerCodigo():entero obtenerSaldo():real	
<<Responsabilidades>> Asegura código>0 y saldo>=-maxDescubierto	

```
//Comandos
public void depositar(float mto){
    /*Incrementa el saldo en mto, Requiere mto > 0*/
    saldo += mto; }

public boolean extraer(float mto){
    /*Requiere mto > 0. Si mto > saldo+maxDescubierto
    extraer retorna false y la extracción no se
    realiza, en caso contrario decrementa el saldo en
    mto y retorna true*/
    boolean puede = true;
    if (saldo + maxDescubierto >= mto)
        saldo=saldo-mto;
    else puede = false;
    return puede; }
```

La variable **mto** declarada como **parámetro formal** se trata como una **variable local**

Ejemplo: Cajero automático

CtaCteBancaria	
<<Atributo de clase>> maxDescubierto = 1000	
<<Atributos de instancia>> codigo:entero saldo:real	
<<Constructores>> CtaCteBancaria(c:entero) CtaCteBancaria(c:entero,s:real)	
<<Comandos>> depositar(mto:real) extraer(mto:real):boolean	
<<Consultas>> obtenerCodigo():entero obtenerSaldo():real	
<<Responsabilidades>> Asegura código>0 y saldo>=-maxDescubierto	

```
// Consultas
public int obtenerCodigo(){
    return codigo; }

public float obtenerSaldo(){
    return saldo; }
```


Ejemplo: Cajero Automático

Convenciones para la implementación

- Declaramos las variables al principio de cada bloque.
- Usamos **identificadores significativos** e **identamos** adecuadamente.
- Incluimos **comentarios** que describan la **estructura** del código, la **funcionalidad** de cada método y las **responsabilidades** establecidas en el diseño.
- **No exageramos** con los comentarios oscureciendo la lógica de la resolución.
- No escribir comentarios que expliquen características del lenguaje
- Si un método produce un resultado, incluimos **una única instrucción de retorno al final**, excepto si todo el código del método es un if-else con instrucciones simples.
- Los atributos de instancia los declaramos **privados**.
- La clase **CtaCteBancaria** no lee ni muestra datos, toda la entrada y salida la hace la clase que usa a **CtaCteBancaria**.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Ejemplo: Cajero Automático

La verificación de CtaCteBancaria

- Durante la implementación de un sistema para un Banco, la clase **CtaCteBancaria** puede pensarse como una pieza, un **módulo** de la colección de módulos que en conjunto va a constituir el sistema.
- Antes de que la clase se integre en la colección y pueda ser usada por otras clases, es muy importante **verificar** que actúa de acuerdo a su especificación.
- Los casos de prueba puede definirlos el responsable de testing o el diseñador, considerando las **responsabilidades** de la clase y las **restricciones** y **funcionalidades** especificadas en los requerimientos.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Ejemplo: Cajero Automático

La verificación de CtaCteBancaria

- Una alternativa es escribir una clase **testerCtaCteBancaria** que verifique los servicios provistos por la clase **CtaCteBancaria** para un conjunto de casos de prueba.
- La clase **testerCtaCteBancaria** **USA** **CtaCteBancaria** e incluye el método **main** que inicia la ejecución.
- La clase desaparece no forma parte del sistema, no modela a ningún objeto del problema

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Ejemplo: Cajero Automático

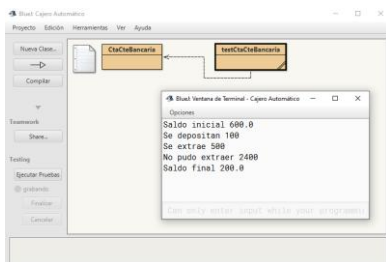
La verificación de CtaCteBancaria

```
class testerCtaCteBancaria {
public static void main (String a[]) {
    CtaCteBancaria cb = new CtaCteBancaria(123,600);
    System.out.println("Saldo inicial "+cb.obtenerSaldo());
    cb.depositar(100);
    System.out.println("Se depositan 100 ");
    if(!cb.extraer(500))
        System.out.println("No pudo extraer 500");
    else
        System.out.println("Se extrae 500");
    if(!cb.extraer(2400))
        System.out.println("No pudo extraer 2400");
    else
        System.out.println("Se extrae 2400");
    System.out.println("Saldo final "+cb.obtenerSaldo()); }
}
```

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Ejemplo: Cajero Automático

La verificación de CtaCteBancaria



```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Ejemplo: Cajero Automático

La verificación de CtaCteBancaria

```
class testerCtaCteBancaria {
public static void main (String a[]) {
    CtaCteBancaria cb = new CtaCteBancaria(123,600);
```

- Declara la variable **cb**
- Crea un objeto de clase **CtaCteBancaria**
- Liga el objeto a la variable **cb**

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Ejemplo: Cajero Automático

La verificación de CtaCteBancaria

```
class testCtaCteBancaria {
    public static void main (String a[]) {
        CtaCteBancaria cb = new CtaCteBancaria(123,600);
    }
}
```

```
public CtaCteBancaria(int c, float s) {
    // Requiere c >0 y s >= 0
    codigo = c; saldo = s; }
}
```

Crea un objeto de clase CtaCteBancaria

- Reserva espacio en memoria
- Ejecuta el constructor de la clase con dos parámetros.

Ejemplo: Cajero Automático

La verificación de CtaCteBancaria

```
class testCtaCteBancaria {
    public static void main (String a[]) {
        CtaCteBancaria cb = new CtaCteBancaria(123,600);
        System.out.println("Saldo inicial "+cb.obtenerSaldo());
    }
}
```

```
// Consultas
public float obtenerSaldo(){
    return saldo;}
}
```

Envía el mensaje obtenerSaldo al objeto ligado a la variable cb.

Ejemplo: Cajero Automático

La verificación de CtaCteBancaria

```
class testCtaCteBancaria {
    public static void main (String a[]) {
        CtaCteBancaria cb = new CtaCteBancaria(123,600);
        System.out.println("Saldo inicial "+cb.obtenerSaldo());
    }
}
```

```
// Consultas
public float obtenerSaldo(){
    return saldo;}
}
```

La consulta obtenerSaldo retorna el valor del atributo de instancia saldo.

Ejemplo: Cajero Automático

La verificación de CtaCteBancaria

```
class testCtaCteBancaria {
    public static void main (String a[]) {
        CtaCteBancaria cb = new CtaCteBancaria(123,600);
        System.out.println("Saldo inicial "+cb.obtenerSaldo());
    }
}
```

El método println recibe como parámetro un objeto de clase String. String es una clase provista por java. Un objeto de clase String es una cadena de caracteres

Ejemplo: Cajero Automático

La verificación de CtaCteBancaria

```
class testCtaCteBancaria {
    public static void main (String a[]) {
        CtaCteBancaria cb = new CtaCteBancaria(123,600);
        System.out.println("Saldo inicial "+cb.obtenerSaldo());
    }
}
```

En este caso el parámetro real de println es una cadena de caracteres formada por la concatenación de un literal y el resultado que retorna luego de que el objeto ligado a la variable cb recibe el mensaje obtenerSaldo. obtenerSaldo retorna un número real, que se convierte a String antes de que se ejecute la concatenación.

Ejemplo: Cajero Automático

La verificación de CtaCteBancaria

```
class testCtaCteBancaria {
    public static void main (String a[]) {
        CtaCteBancaria cb = new CtaCteBancaria(123,600);
        System.out.println("Saldo inicial "+cb.obtenerSaldo());
        cb.depositar(100);
    }
}
```

```
//Comandos
public void depositar(float mto){
    // Requiere mto > 0
    saldo += mto; }
}
```

Se envía el mensaje depositar al objeto ligado a la variable cb.

Ejemplo: Cajero Automático

La verificación de CtaCteBancaria

```
class testCtaCteBancaria {
    public static void main (String a[]) {
        CtaCteBancaria cb = new CtaCteBancaria(123,600);
        System.out.println("Saldo inicial "+cb.obtenerSaldo());
        cb.depositar(100);

        //Comandos
        public void depositar(float mto){
            // Requiere mto > 0
            saldo += mto; }
    }
```

La ejecución del comando **depositar** modifica el valor del atributo de instancia **saldo** del objeto que recibió el mensaje.

Ejemplo: Cajero Automático

La verificación de CtaCteBancaria

```
class testCtaCteBancaria {
    public static void main (String a[]) {
        CtaCteBancaria cb = new CtaCteBancaria(123,600);
        System.out.println("Saldo inicial "+cb.obtenerSaldo());
        cb.depositar(100);
        System.out.println("Se depositan 100 ");
    }
```

Se muestra en consola la cadena de caracteres "Se depositan 100 "

Ejemplo: Cajero Automático

La verificación de CtaCteBancaria

```
class testCtaCteBancaria {
    public static void main (String a[]) {
        CtaCteBancaria cb = new CtaCteBancaria(123,600);
        System.out.println("Saldo inicial "+cb.obtenerSaldo());
        cb.depositar(100);
        System.out.println("Se depositan 100 ");
        if(!cb.extraer(500))

        public boolean extraer(float mto){
            /*Requiere mto > 0. Si mto>saldo+maxDescubierto extraer retorna false y la extracción
            no se realiza, en caso contrario decrementa el saldo en mto y retorna true*/
            boolean puede = true;
            if (saldo + maxDescubierto >= mto)
                saldo=saldo-mto;
            else puede = false;
            return puede;}
    }
```

Envía el mensaje **extraer** al objeto ligado a la variable **cb**.

Ejemplo: Cajero Automático

La verificación de CtaCteBancaria

```
class testCtaCteBancaria {
    public static void main (String a[]) {
        CtaCteBancaria cb = new CtaCteBancaria(123,600);
        System.out.println("Saldo inicial "+cb.obtenerSaldo());
        cb.depositar(100);
        System.out.println("Se depositan 100 ");
        if(!cb.extraer(500))

        public boolean extraer(float mto){
            /*Requiere mto > 0. Si mto>saldo+maxDescubierto extraer retorna false y la extracción
            no se realiza, en caso contrario decrementa el saldo en mto y retorna true*/
            boolean puede = true;
            if (saldo + maxDescubierto >= mto)
                saldo=saldo-mto;
            else puede = false;
            return puede;}
    }
```

El método **extraer** retorna **true** y modifica el valor del atributo **saldo**.

Ejemplo: Cajero Automático

La verificación de CtaCteBancaria

```
class testCtaCteBancaria {
    public static void main (String a[]) {
        CtaCteBancaria cb = new CtaCteBancaria(123,600);
        System.out.println("Saldo inicial "+cb.obtenerSaldo());
        cb.depositar(100);
        System.out.println("Se depositan 100 ");
        if(!cb.extraer(500))
            System.out.println("No pudo extraer 500");
        else
            System.out.println("Se extraen 500");
    }
```

Cuando el control retorna al método **main** se ejecuta el condicional y como el valor de verdad es **false**, se muestra en consola una cadena de caracteres "Se extraen 500".

Los constructores

- Un **constructor** es un **servicio** provisto por la clase y se caracteriza porque recibe el **mismo nombre que la clase**.
- El constructor **se invoca cuando se crea un objeto** y habitualmente se usa para **inicializar** los valores de los atributos de instancia.
- Una clase puede brindar **varios constructores**, siempre que tengan diferente número o tipo de parámetros.
- Si en una clase no se define explícitamente un constructor, el compilador crea automáticamente uno.
- Si la clase incluye uno o más constructores, el compilador no agrega ningún otro implícitamente.

Los comandos y las consultas

- Los **comandos** son servicios que modifican los valores de los atributos de un objeto.
- Las **consultas** son servicios que no modifican los valores de los atributos, en general, devuelven un resultado que corresponde al valor de un atributo o al cómputo de una expresión.
- Un comando puede retornar también un valor.
- Los comandos y consultas conforman los **métodos** de una clase.
- Con frecuencia una clase incluye un comando para modificar cada atributo de instancia y una consulta para retornar el valor de cada atributo de instancia. Los llamamos **comandos y consultas triviales**.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```



En esta clase

- El modelo computacional de la POO
- El diagrama de una clase
- La implementación de una clase en Java
- Los atributos de clase y de instancia
- Los servicios: constructores, comandos y consultas
- Ejemplo: Cajero automático
- La clase tester

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```



En la próxima clase

- Pueden **leer** el diagrama de una clase.
- Pueden **implementar** en Java una clase con atributos de tipo elemental y servicios que reciben y retornan datos de tipo elemental.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

